

**DIRE DAWA UNIVERSITY
INSTITUTE OF TECHNOLOGY
SCHOOL OF COMPUTING
DEPARTMENT OF SOFTWARE ENGINEERING**

SENIOR PROJECT DEFENSE

Design and Implementation of Temariware: A Telegram Mini App for University Student Job Matching in Ethiopia

By: Nebiyu Tsegaye

ID: 1501332

Advisor: [Advisor Name (Title)]

Date: [Month, Year]

Dire Dawa, Ethiopia

Outline

1. Introduction & Background
 2. Problem Statement
 3. Objectives
 4. Scope & Limitations
 5. Methodology & Tools
 6. Significance
1. Work Breakdown & Feasibility
 2. System Requirements
 3. System Analysis
 4. Software Architecture
 5. Detailed Design
 6. Conclusion & Future Work

Introduction & Background

- Telegram is the dominant messaging platform for Ethiopian university students, with millions of daily active users.
- Informal job channels (Afriwork, Yegna Tutors Harrar) post tutoring, freelance, and part-time roles daily.
- These channels are free, instant, and reach a large targeted audience.
- However, posts are unstructured free text with no filtering, no archive, and no verification.
- Temariware proposes a structured Telegram Mini App + Bot to solve these gaps.
- Combines the reach of Telegram with the structure of a traditional job-matching platform.

Statement of the Problem

- Job posts are unstructured free text — impossible to filter by subject, location, or salary.
- No persistent archive — opportunities disappear from chat history within hours.
- Neither employers nor students are verified — scams and no-shows are common.
- Application process is highly manual — students forward CVs repeatedly, no tracking.
- No payment mechanism — channels cannot sustain themselves or filter serious applicants.
- Existing platforms (EthioJobs, Ezega) focus on full-time graduate roles, not student part-time work.

General Objective

“To design, develop, and deploy a Telegram Mini App and Bot called Temariware that connects Ethiopian university students with verified tutoring, freelance, part-time, and full-time job opportunities, while providing integrated payment verification, mandatory document checks, and automated hiring workflow management.

3

User Roles
(Student, Employer, Admin)

3

Languages
(EN, Amharic, Afaan Oromoo)

0 ETB

Monthly Cost
(Free-Tier Infrastructure)

Specific Objectives

OBJECTIVES 1 — 5

1. Analyse limitations of existing informal Telegram job channels and elicit stakeholder requirements.
2. Design a multi-section Mini App UI (job feed, detail, post, applications, profile, admin) with 3-language support.
3. Design and implement a PostgreSQL database schema via Prisma ORM (users, jobs, applications, payments, subscriptions, verifications, settings).
4. Implement a Telegram Bot webhook handling 9 slash commands and pushing instant alerts.
5. Implement Telegram WebApp initData HMAC-SHA256 authentication

OBJECTIVES 6 — 10

1. Implement screenshot-based payment verification (job post fee + per-job application fee).
2. Implement mandatory document upload (National ID, University ID, grade report) with image compression.
3. Implement automatic job closure when positionsAvailable is reached.
4. Deploy on free-tier infrastructure (Vercel + Neon Postgres + Telegram Bot API).
5. Document the system per the Dire Dawa University industrial project guideline.

Scope of the Project

IN SCOPE

- Next.js Mini App with 6 primary views
- Telegram bot with 9 slash commands
- PostgreSQL database with 7 entities
- HMAC-based authentication layer
- Screenshot-based payment verification
- Mandatory document upload with compression
- Per-job configurable application fees
- Automatic job closure on hiring target
- Channel broadcast to @TEMARIWARE
- Per-subscriber push alerts

OUT OF SCOPE (V2)

- Real Chapa / Telebirr API integration
- Automated scraping of existing channels
- In-app CV file storage (uses external links)
- Email or SMS notifications
- Employer dashboard with multi-job analytics
- Saved/bookmarked jobs
- Reporting beyond basic admin stats

Limitations

- Payment verification is manual (screenshot review) — introduces hours of latency, single point of failure.
- System depends entirely on Telegram — no Telegram account, no access; outages affect availability.
- Images stored as base64 in PostgreSQL (not object storage) — 1MB practical limit per image.
- No automated authenticity checks on uploaded IDs — relies on admin visual inspection.
- Currently supports only the Ethiopian market and three languages.
- No in-app dispute resolution mechanism — handled via external Telegram chat.

Methodology & Approach

- Object-Oriented Software Development approach (per university guideline).
- Iterative and incremental development process — each iteration produces a working slice.
- Loosely based on the Agile Unified Process — lightweight upfront design + continuous integration.
- Five phases: Requirements, Design, Implementation, Testing, Deployment.
- Each phase ends with advisor review milestone for continuous feedback.
- Continuous deployment to Vercel staging environment throughout development.

Techniques & Tools

TECHNIQUES

- Object-Oriented Analysis (entities, relationships)
- Use Case Modelling (actors + use cases)
- CRC Modelling (class-responsibility-collaboration)
- Sequence & Activity Diagrams
- Client-side image compression (Canvas API)
- HMAC-SHA256 cryptographic validation

TOOLS

- Next.js 16 with App Router
- React 19 + TypeScript 5
- Tailwind CSS 4 + shadcn/ui
- Prisma ORM 6 + Neon Postgres
- Vercel (deployment + CDN)
- Telegram Bot API
- Git + GitHub (version control)

Significance of the Project

- **For Students:** reduces friction of finding part-time work; subscribe to subject/location alerts; apply with one tap using pre-verified profile; mandatory documents build trust with employers.
- **For Employers:** structured applicant tracking; per-job application fee filters casual browsers; review documents in one panel; accept/reject with one tap; auto-close positions when hiring target met.
- **For the Ecosystem:** reference implementation of a production Telegram Mini App on free-tier infrastructure; demonstrates youth entrepreneurship without upfront capital; formalises a previously informal labour market.
- Creates data on youth employment patterns for future research.
- Sustainable revenue model through configurable fees.
- Multilingual (EN/AM/OR) ensures accessibility across Ethiopia.

Work Breakdown Structure

ID	Work Package	Duration	Deliverables
WP1	Requirements Elicitation & Analysis	2 weeks	Requirements doc, use cases
WP2	System Design	2 weeks	Schema, API contracts, UI mockups
WP3	Database & Backend Implementation	3 weeks	Prisma schema, API routes, auth
WP4	Frontend Mini App Implementation	3 weeks	All views, navigation, i18n
WP5	Bot Webhook & Channel Integration	1 week	Bot commands, broadcasts, alerts
WP6	Testing, Deployment, Documentation	1 week	Deployed app, this document

Total: ~16 person-weeks across 4 team members over 12 calendar weeks.

Feasibility Analysis

Dimension	Analysis	Verdict
Technical	All tools are free, open-source, well-documented. Team has relevant coursework background.	Feasible
Economic	Zero monthly recurring cost (Vercel free + Neon free + Telegram Bot API free). Only cost is team time.	Feasible
Operational	Aligns with existing Telegram usage habits. No user training required. Document verification is intuitive.	Feasible
Schedule	12-week timeline with 4 team members. Iterative development with continuous integration.	Feasible (tight)

Main risk: single administrator for payment verification — mitigated in v2 via Chapa API integration.

Current System Description

- Informal Telegram channels (Afriwork, Yegna Tutors Harrar) broadcast job posts as free text.
- Employer composes a free-text message and forwards to channel operator's bot.
- Operator manually reviews, reformats, and publishes to the channel.
- Interested students forward the post + CV to employer's personal Telegram.
- Employer reviews applications in private chat, contacts candidates directly.
- Payments (when required) made via Telebirr with no formal record.

The current "system" is a manual, chat-based workflow — fast and familiar, but with no structure, no archive, and no audit trail. Temariware preserves the speed while adding the missing scaffolding.

Functional Requirements (Key)

FR-01 TO FR-05

FR-01: Authenticate users via Telegram initData HMAC-SHA256

FR-02: Browse approved jobs without authentication

FR-03: Filter jobs by subject, location, type, gender, search

FR-04: Upload National ID, University ID, grade report with compression

FR-05: Block applications from students with incomplete profiles

FR-06 TO FR-10

FR-06: Verified employers post jobs with all metadata

FR-07: Require payment screenshot when jobPostFee > 0

FR-08: Admins approve/reject pending jobs with reason

FR-09: Broadcast approved jobs to @TEMARIWARE channel

FR-10: Push alerts to subscribers matching job category

Non-Functional Requirements

ID	Category	Requirement
NFR-01	Performance	API response < 500ms (95th percentile); Mini App first paint < 2s on 3G
NFR-02	Security	HMAC-SHA256 auth; HTTPS only; no plaintext payment data
NFR-03	Usability	360px-wide screen support; 3 languages; 44px touch targets; haptic feedback
NFR-04	Reliability	99% uptime (Vercel SLA); graceful error handling
NFR-05	Scalability	Serverless scales to 1000 concurrent users on free tier
NFR-06	Maintainability	TypeScript throughout; modular components; lint-clean
NFR-07	Compatibility	Telegram mobile (iOS/Android), Desktop, Web; modern browser fallback

Business Rules & Constraints

KEY BUSINESS RULES

- Only verified employers or admins can post jobs.
- Students cannot apply until profile is complete (3 documents uploaded).
- One application per student per job (unique constraint).
- Job posting requires payment screenshot (default 500 ETB).
- Application fees are per-job configurable by employer.
- Auto-close when filledPositions reaches positionsAvailable.

KEY CONSTRAINTS

- Free-tier infrastructure only (zero recurring cost).
- Screenshot-based payment (no Chapa API in v1).
- Images stored as base64 in PostgreSQL (1MB max).
- Mini App runs inside Telegram WebApp container.
- No web_app buttons in channel posts (only URL buttons).
- Three languages with runtime switching.

Essential Use Case Model

Four primary actors, fifteen use cases.

STUDENT

Browse Job Feed, View Job Detail, Complete Profile, Apply to Job, Subscribe to Alerts, Upload Payment, View My Applications

EMPLOYER

Post Job, Review Applicants, Accept Applicant, Reject Applicant, Verify Employer

ADMINISTRATOR

Approve Job, Approve Payment, Approve Verification, Configure Settings

TELEGRAM BOT

Handle Slash Commands, Push Job Alerts, Broadcast to Channel

Include relationship: **Apply to Job** includes **Check Profile Complete**. **Extend** relationship: **Post Job** extends **Upload Payment Screenshot** (only when fee > 0). Full use case diagram in Chapter 2, Figure 2.7.

Conceptual Class Model

- **Seven primary classes:** User, Job, Application, Subscription, VerificationRequest, Setting, Payment.
- A **User** has many Jobs (as employer), many Applications (as student), many Subscriptions, and may have one VerificationRequest.
- A **Job** has many Applications and may have associated Payments.
- An **Application** belongs to one User and one Job, with a unique constraint on (jobId, userId).
- A **Setting** is a singleton (single row) storing global configuration.
- **Payment** is an audit log linking a User, Job, and optionally an Application.

Full class diagram with attributes and operations in Chapter 3, Figure 3.5. Complete Prisma schema in Appendix C.

Sequence & Activity Diagrams

SEQUENCE DIAGRAMS

- **Apply to Job:** Mini App → Jobs API → Applications API → Database → Employer (Telegram notification)
- **Accept Applicant and Pay:** Employer → Accept API → Student (notification) → Pay API → Admin → Approve Payment API
- **Approve Job and Broadcast:** Admin → Approve API → Channel (Telegram Bot API) → Subscribers (push)

ACTIVITY DIAGRAMS

- **Job Posting & Approval Flow:** submit → payment screenshot → admin review → approve/reject → broadcast → alerts
- **Application Lifecycle:** PENDING → ACCEPTED → PAYMENT_PENDING → HIRED (or REJECTED/CLOSED)
- **Auto-close trigger:** when filledPositions = positionsAvailable, mark job CLOSED, notify all parties

Full diagrams in Chapter 2, Figures 2.9–2.13.

Software Architecture

- **Selected style:** Layered Architecture with four layers — Presentation, Application, Domain, Persistence.
- **Presentation Layer:** React 19 components (Mini App views).
- **Application Layer:** Next.js API routes (controllers).
- **Domain Layer:** Prisma models + business logic.
- **Persistence Layer:** PostgreSQL on Neon (Frankfurt, eu-central-1).
- **Chosen for** clear separation of concerns, Next.js convention alignment, team familiarity.

Alternatives considered: Microkernel (too complex), Event-Driven (unnecessary complexity), Serverless-First (already adopted at deployment). Full comparison in Chapter 3, Table 3.1.

Component & Deployment

COMPONENTS

- Next.js Mini App (Vercel) — frontend + API routes
- Telegram Bot Webhook (within Next.js) — receives updates
- PostgreSQL Database (Neon Frankfurt) — data persistence
- Telegram Bot API (external) — message sending + initData
- @TEMARIWARE Channel (external) — public job broadcast

DEPLOYMENT NODES

- Vercel Edge Network — runs Next.js globally
- Neon Cloud (Frankfurt) — PostgreSQL, closest to Ethiopia
- Telegram Cloud — Bot API + channel hosting
- End users access via Telegram client (mobile/desktop/web)
- Webhook: Telegram Cloud → Vercel via HTTPS

Persistent Data Model

Table	Key Fields
User	id, telegramId, role, language, profile + document fields, verification, isAdmin
Job	id, title, subjects, location, workType, salary, positionsAvailable, applicationFee, status
Application	id, jobId, userId, message, status, paymentScreenshot, hiredAt
Subscription	id, userId, category
VerificationRequest	id, userId, companyName, contactTelegram, status
Setting	id (singleton), jobPostFee, applicationFee, paymentInstructions
Payment	id, userId, jobId, type, amount, screenshotUrl, status, reviewedBy

Full ER diagram: Chapter 3, Figure 3.4. Complete Prisma schema: Appendix C.

Detailed Design

- **Design-Level Class Diagram:** refines conceptual model with attributes, operations, visibility (Figure 3.5).
- **Collaboration Diagrams:** Apply to Job, Post Job and Pay, Approve Payment and Hire (Figures 3.6–3.8).
- **State Chart Diagram:** Application lifecycle — PENDING → ACCEPTED → PAYMENT_PENDING → HIRED (or REJECTED/CLOSED) (Figure 3.9).
- **User Interface Diagrams:** 6 views — Home, Job Detail, Post Job, Applications, Profile, Admin (Figures 3.10–3.15).
- **Class Type Architecture:** Entity (7 classes), Boundary (7 classes), Control (6 classes) — ECB pattern.
- All design artefacts provide a complete blueprint for implementation handoff.

Implementation Status

LIVE DEPLOYMENT

- Bot: **@temariwarebot** (live on Telegram)
- Channel: **@TEMARIWARE** (broadcasts working)
- Mini App: deployed on Vercel (temariware-rouge.vercel.app)
- Database: Neon Postgres (Frankfurt)
- Webhook: configured and verified
- All 9 bot commands functional

IMPLEMENTED FEATURES

- 6 Mini App views with 3-language support
- Job posting with payment screenshot
- Application with mandatory documents
- Employer accept/reject applicants
- Student payment upload after acceptance
- Admin payment approval + auto-hire
- Auto-close on positions filled
- Channel broadcast + subscriber alerts

System is production-ready and accepting real users.

Conclusion & Future Work

CONCLUSION

- Addresses structural gaps in Ethiopia's informal Telegram job market
- All 10 specific objectives achieved
- System deployed, live, accepting real users on free-tier infrastructure
- Combines job matching, document verification, and payment workflow
- Demonstrates zero-cost production-grade software in Ethiopia

FUTURE WORK (V2)

- Integrate Chapa API for automated payment verification
- Migrate image storage to Supabase Storage / S3
- Add in-app messaging between students and employers
- Automated scraping from existing Telegram channels
- Employer dashboard with multi-job analytics
- Expand to other East African markets

Thank You

Questions & Answers

Nebiyu Tsegaye

ID: 1501332

Department of Software Engineering

Dire Dawa University

@temariwarebot on Telegram